

1. Kursarbeit: Vorbereitung, Kompetenzen & Handout

Deklarative Programmierung in Prolog: Fakten, Inferenzregeln, Datenfluss & Rekursion

Prüfungsleitfaden
90 Minuten

Dauer: 90 Min. (Doppelstunde)

Termin: Nächste Woche

Hilfsmittel: Stift, Konzeptpapier (keine Spickzettel)

Aufbau der Arbeit: Die Kursarbeit deckt die drei Anforderungsbereiche ab: **AFB I** (Wissen wiedergeben & Syntax), **AFB II** (Modellieren von Regeln, Datenfluss & Tracing-Tabellen) sowie **AFB III** (Fehlerdiagnose / Debugging von Rekursionsfällen & Transfer).

1 Offizielle Kompetenzenliste („Ich kann...“) zur Prüfungsvorbereitung

Status	Kompetenzbereich	Konkrete Kompetenzbeschreibung („Ich kann...“)
☐	1. Fachbegriffe & Grundlagen Begriffs-TÜV	Ich kenne und erkläre die zentralen Fachbegriffe der logischen Programmierung: Fakten , Atome (beginnen klein), Variablen (beginnen groß), Regeln (<code>: -</code>), Prädikate & Stelligkeit (z. B. <code>vater/2</code>), Inferenz (automatisches logisches Schließen) sowie die Closed World Assumption (CWA) .
☐	2. Wissensbasis anlegen & erweitern Fakten & Anfragen	Ich kann reale Sachverhalte und Beziehungen fehlerfrei als logische Fakten deklarieren, eine bestehende Wissensbasis gezielt erweitern und durch präzise Ja/Nein-Anfragen sowie Variablen-Anfragen (samt Backtracking mit <code>;</code>) abfragen und die Ausgabe vorhersagen.
☐	3. Regeln erstellen & anwenden Inferenz & Datenfluss	Ich kann eigene Regeln mit Kopf und Körper formulieren, logisches UND (Komma <code>,</code>) und logisches ODER (mehrere Regeln) einsetzen, den Datenfluss über Zwischenvariablen (Stafetten-Prinzip) steuern und Schutzschranken (<code>\=</code>) sowie die anonyme Variable (<code>_</code>) gezielt platzieren.
☐	4. Rekursion Beliebige Tiefen & Netze	Ich kann das 2-Säulen-Prinzip (Basisfall & Rekursionsschritt) anwenden, um beliebige Stammbaumtiefen oder Netzwerkpfade zu durchlaufen, Anfragen in Tracing-Tabellen schrittweise nachvollziehen, Linksrekursion (Stack Overflow) vermeiden und das Prinzip auf neue Netze übertragen.

Begriffs-Check: **Fakten** sind das Grundwissen • **Regeln** sind die Denkgesetze • **Inferenz** ist der automatische Denkprozess der Engine, der aus beidem neues Wissen ableitet!

Lerntipp für die Vorbereitung: Gehe diese vier Bereiche durch. Wenn du bei einem Punkt unsicher bist, nutze das interaktive Webmodul auf dem iPad oder teste den Code direkt in SWISH!

Das kompakte Prolog-Merkblatt (Kernkonzepte)

2 Syntax-Grundregeln & Vokabular

Atome vs. Variablen

Syntax

- **Atome (Konstanten):** Beginnen zwingend mit Kleinbuchstaben: `zeus`, `ares`, `diff10`.
- **Variablen (Platzhalter):** Beginnen zwingend mit Großbuchstaben: `X`, `Wer`, `Kind`.
- **Prädikate & Stelligkeit:** Name klein, Anzahl Argumente in Klammern: `vater/2`, `weiblich/1`.

Operatoren & Satzzeichen

Logik

- `:-` bedeutet „WENN“ (trennt Regelkopf von Körper).
- Komma `,` bedeutet logisches **UND** (Konjunktion).
- Mehrere Regeln mit gleichem Kopf bedeuten logisches **ODER**.
- Jeder Satz endet mit einem **Punkt** `.`!

3 Die 3 Goldenen Inferenz-Gesetze

1. Stafetten-Gesetz

Datenfluss über gleiche Variablen:

```
grossvater(G, K) :-
    vater(G, E),
    elternteil(E, K).
```

Variable `E` bindet bei Ziel 1 und übergibt den Wert als Stafette an Ziel 2.

2. Schranken-Gesetz

Schutz vor Selbstreferenz (`\=`):

```
geschwister(X, Y) :-
    elternteil(E, X),
    elternteil(E, Y),
    X \= Y.
```

Operator `\=` steht immer am Ende, wenn X und Y gebunden sind!

3. Scheuklappen-Gesetz

Anonyme Variable (`_`):

```
hat_kinder(X) :-
    elternteil(X, _).
```

Name des Kindes ist egal – nur die Existenz zählt. Jeder Unterstrich ist unabhängig!

4 Das 2-Säulen-Prinzip der Rekursion

Säule 1: Der Basisfall (Abbruchbedingung)

„Jeder direkte Elternteil ist sofort auch Vorfahre.“

```
vorfahre(X, Y) :- elternteil(X, Y).
```

Säule 2: Der Rekursionsschritt (Kette / Staffelstab)

„X ist Vorfahre von Y, wenn X Elternteil von Z ist UND Z Vorfahre von Y ist.“

```
vorfahre(X, Y) :- elternteil(X, Z),
    vorfahre(Z, Y).
```

- **Kettenglied-Modell:** $X \xrightarrow{\text{elternteil}} Z \xrightarrow{\text{vorfahre (Restkette)}} Y$. In jedem Schritt wird das Problem um eine Generation kleiner!

MERKE

Die zwei tödlichen Klausurfällen (Fehlervermeidung!)

1. Linksrekursion (Left Recursion):

```
vorfahre(X, Y) :- vorfahre(X, Z), elternteil(Z, Y).
```

Fehler: Prolog ruft sich sofort wieder selbst auf, bevor `Z` gebunden ist → **Endlosschleife / Stack Overflow!**

- Die eiserne Klausur-Regel: Im Rekursionsschritt bindet das Fakten-Prädikat (`elternteil(X, Z)`) **immer zuerst** die Variable `Z`, bevor der rekursive Aufruf folgt!

2. Vergessener Basisfall:

Wer Säule 1 weglässt, verhindert das Erkennen direkter Elternbeziehungen → direkte Anfragen liefern fälschlicherweise **false.**!

Aufgabe 1: Fakten, Wissensbasen & Closed World Assumption

AFB I • Grundlagen

Gegeben ist folgende Wissensbasis eines Sportvereins:

```
mitglied(anna, tennis).
mitglied(ben, fußball).
mitglied(clara, tennis).
trainer(huber, tennis).
trainer(meier, fußball).
```

- a) Was antwortet Prolog auf die Anfrage `?- mitglied(clara, tennis).`?
- b) Was liefert die Anfrage `?- mitglied(Wer, tennis).` bei Tastendruck auf `;`?
- c) Erkläre, warum die Anfrage `?- mitglied(felix, schach).` mit `false.` beantwortet wird, obwohl Felix tatsächlich Schach spielt. Welches Prinzip liegt vor?

MUSTERLÖSUNG AUFGABE 1:

- a) **true.** (Der Fakt existiert wortwörtlich in der Wissensbasis).
- b) 1. Treffer: **Wer = anna** ; 2. Treffer: **Wer = clara.** (Backtracking durchsucht alle Fakten).
- c) Prolog antwortet mit **false.** aufgrund der **Closed World Assumption (CWA)** (Geschlossene-Welt-Annahme). Prolog geht davon aus, dass alles, was nicht in der Wissensbasis explizit steht oder logisch daraus abgeleitet werden kann, in dieser Welt nicht wahr (falsch) ist.

Aufgabe 2: Regeln modellieren (Stafette, Schutzschranke & Scheuklappe)

AFB II • Modellierung

Erweitere die obige Vereins-Wissensbasis um drei neue Regeln. Beachte die Regeln für Datenfluss und Schranken:

- a) Formuliere die Regel `trainingspartner(A, B)`: Zwei Personen A und B sind Trainingspartner, wenn sie dasselbe Mitgliedsfach belegen, aber nicht dieselbe Person sind.
- b) Formuliere die Regel `trainer_von(T, S)`: T ist Trainer von S , wenn T dieselbe Sportart trainiert, die S als Mitglied betreibt (nutze eine Zwischenvariable).
- c) Formuliere die Regel `ist_sportlich(X)` unter Verwendung der **anonymen Variable** `_`: Wahr, wenn X irgendeine Sportart betreibt.

MUSTERLÖSUNG AUFGABE 2:

- a) Schutzschranke `\=` verhindert, dass Anna ihre eigene Trainingspartnerin ist:

```
trainingspartner(A, B) :-
    mitglied(A, S),
    mitglied(B, S),
    A \= B.
```

- b) Datenfluss über die Zwischenvariable **Sport**:

```
trainer_von(T, S) :-
    trainer(T, Sport),
    mitglied(S, Sport).
```

- c) Anonyme Variable `_` ignoriert den konkreten Sportartnamen:

```
ist_sportlich(X) :-
    mitglied(X, _).
```

Aufgabe 3: Algorithmisches Tracing-Protokoll der Rekursion

AFB II • Tracing

Gegeben sind die 2 Säulen der Rekursion und folgende Fakten:

```
elternteil(gaia, kronos).   elternteil(kronos, zeus).   elternteil(zeus, herakles).
vorfahre(X, Y) :- elternteil(X, Y).
vorfahre(X, Y) :- elternteil(X, Z), vorfahre(Z, Y).
```

Ein Benutzer stellt die Anfrage: `?- vorfahre(gaia, herakles).`

Protokolliere den Ablauf der Inferenz-Engine Schritt für Schritt in der Tracing-Tabelle:

Ebene / Aufruf	Was prüft Prolog? (Regel & Teilziel)	Ergebnis & Variablenbindung
Ebene 1: <code>vorfahre(gaia, herakles)</code>	1. Säule 1 (Basisfall): <code>elternteil(gaia, herakles)</code> . 2. Säule 2 (Rekursion): Suche Kind <code>Z</code> : <code>elternteil(gaia, Z)</code> .	1. Scheitert (false) 2. Treffer: <code>Z = kronos</code> → Neuer Aufruf: <code>vorfahre(kronos, herakles)</code>
Ebene 2: <code>vorfahre(kronos, herakles)</code>	1. Säule 1 (Basisfall): <code>elternteil(kronos, herakles)</code> . 2. Säule 2 (Rekursion): Suche Kind <code>Z</code> : <code>elternteil(kronos, Z)</code> .	1. Scheitert (false) 2. Treffer: <code>Z = zeus</code> → Neuer Aufruf: <code>vorfahre(zeus, herakles)</code>
Ebene 3: <code>vorfahre(zeus, herakles)</code>	1. Säule 1 (Basisfall): <code>elternteil(zeus, herakles)</code> . (Greift der Basisfall bereits?)	Erfolg! Treffer als Fakt in Wissensbasis! Säule 1 liefert: true . (Basisfall greift!)
Rücklauf: Synthese	Die Erfolgsmeldung wandert den Aufruf-Stack nach oben zurück (Ebene 3 → Ebene 2 → Ebene 1).	Gesamtergebnis: true . Beweiskette: gaia → kronos → zeus → herakles

Aufgabe 4: Fehlerdiagnose & Rekursions-Transfer (Netzwerke)

AFB III • Transfer & Debugging

a) **Fehlerdetektive:** Ein Schüler notiert zur Wegfindung im Netzwerk:

```
verbunden(A, B) :- verbunden(A, Z), leitung(Z, B).
```

Benenne den Fehler, erkläre das Speicherverhalten beim Ausführen und korrigiere den Code.

b) **Transfer:** Modellierung eines Router-Netzwerks mit Fakten: `kabel(r1, r2). kabel(r2, r3). kabel(r3, r4).`

Formuliere das vollständige rekursive Prädikat `erreichbar(Von, Nach)` nach dem 2-Säulen-Prinzip.

MUSTERLÖSUNG AUFGABE 4:

a) **Fehleranalyse:** Es liegt eine unzulässige **Linksrekursion (Left Recursion)** vor. Da Prolog Ziele strikt von links nach rechts auswertet, ruft sich `verbunden(A, Z)` endlos selbst auf, ohne dass die Variable `Z` vorher gebunden oder das Problem verkleinert wurde. Der Aufrufstapel füllt sich unendlich bis zum Absturz (**ERROR: Out of local stack**).

Korrektur: Der Faktenschritt muss links stehen:

```
verbunden(A, B) :- leitung(A, Z), verbunden(Z, B). (zusätzlich Basisfall nötig!)
```

b) **Vollständiges 2-Säulen-Prädikat für das Router-Netzwerk:**

```
% Säule 1: Basisfall (Direktes Kabel)
erreichbar(Von, Nach) :-
    kabel(Von, Nach).
```

```
% Säule 2: Rekursionsschritt (Verbindung über Zwischen-Router Z)
erreichbar(Von, Nach) :-
    kabel(Von, Z),
    erreichbar(Z, Nach).
```